

Domain-Constrained Chatbot Guardrails

Abhishek Sharma, Chang Zhou, Kautsya Kanu, Keshav Handa, Nabanita Bag, Naveen Murthy
Code & data: <https://github.com/abhishek022k/Domain-Constrained-Chatbot-Guardrails>

1. Introduction

Deployed LLM chatbots routinely break out of the domains they're supposed to stay in. Amazon Rufus, the company's shopping assistant, will write you a Python web-scraper if you ask politely [Lasso Security, 2026]. Translate the same request into Zulu and the safety filter fails roughly 80% of the time, because alignment fine-tuning was done in English only [Deng et al., 2024]. GitHub Copilot's safety filter can be defeated by prepending the word "Sure," to a disallowed request [OWASP, 2025]. They are well-documented, easy to reproduce, and largely unpatched.

The cost of these failures is significant. A customer-support chatbot that writes malware exposes the deploying company to compliance, security and reputational risk. A medical assistant providing legal advice creates liability. As LLMs are deployed into more narrowly scoped, customer-facing roles, the question of how to keep a chatbot on-topic becomes a practical engineering problem with no clear consensus solution.

In this work we treat domain-scope enforcement as a design problem with several reasonable answers, and we run the comparison to find out which one wins. We implement five guardrail strategies that intervene at different points in the LLM pipeline - input filtering, instruction-level prompting, internal-representation probing, parameter fine-tuning, and output critique - and benchmark them on a shared evaluation set. The goal is not to find the single best technique but to map the cost-accuracy frontier so deployers can pick the right tool for their constraints.

2. Hypothesis and Research Question

Research question

Where in the LLM pipeline is domain-scope enforcement most effective: at the input (text classification), via prompted instructions, via internal representations, via parameter fine-tuning, or via output critique?

Strategy	Method	Key idea	Reference papers
S1	Text Classifier	Binary classifier on output generated by LLM	P1 , P2 , P3
S2	Prompted LLM + Self-Reminder	System prompt rules, few-shot, CoT; Constitutional AI style	https://arxiv.org/abs/2310.06117
S3	Representation Probe	Linear probe on residual-stream activations per layer	https://arxiv.org/abs/2310.01405
S4	LoRA fine-tuning(QLoRA)	Train refusal as native behaviour via parameter updates	https://arxiv.org/abs/2106.09685
S5	Critique loop	A model critiques the output and revises it before returning	https://arxiv.org/abs/2303.17651

Hypothesis

Strategies operating on a model's internal representations (S3) or on its parameters via fine-tuning (S4) will outperform surface-level interventions (S1, S2) and output-side interventions (S5) on guardrail accuracy and

false-positive rate. Specifically, we hypothesize that domain-scope is encoded as a **linearly decodable feature** in middle transformer layers, making lightweight linear probes a viable and efficient real-time guardrail.

3. Related Work

Our work sits at the intersection of three research threads.

Off-topic and jailbreak attacks. Recent work documents the systematic failure of deployed safety mechanisms. Lasso Security demonstrated full system-prompt extraction and trivial off-topic compliance in Amazon Rufus [Lasso Security, 2026]. Deng et al. (2024) showed that translating jailbreak prompts into low-resource languages bypasses English-only alignment fine-tuning, with success rates near 80%. Yong et al. (2023) extended this to GPT-4-class models. OWASP's 2025 roundup documents automated frameworks achieving up to 99% jailbreak success across GPT-4o, Gemini 2.0, and DeepSeek-V3. These attacks target safety guardrails (harmful content); our work treats the more practical and underexplored question of keeping a chatbot within its declared business scope.

Surface-level and instruction-based defenses. Constitutional AI [Bai et al., 2022] uses self-generated critiques to steer outputs toward declared principles. Self-Reminder [Xie et al., 2023] prepends and appends safety instructions around user inputs. Llama Guard [Inan et al., 2023] and NVIDIA NeMo Guardrails are production safety systems operating primarily at the input/output text level. These methods are widely deployed but, as the attack literature shows, are often defeated by superficial perturbations. None specifically target domain-scope enforcement.

Internal-representation methods. Probing classifiers have a long history in NLP interpretability [Alain & Bengio, 2016; Belinkov, 2022]. Representation Engineering [Zou et al., 2023] showed that high-level concepts such as truthfulness and harmfulness are linearly decodable from transformer activations, and that interventions in this representation space can steer model behavior. Our S3 strategy applies the same probing methodology to the previously unexamined question of domain-scope enforcement.

Fine-tuning approaches. LoRA [Hu et al., 2022] and QLoRA enable parameter-efficient fine-tuning. Fine-tuning is the standard approach to teaching a model new behavior but modifies base parameters and requires retraining whenever the desired behavior changes. Our S4 strategy uses QLoRA to test whether refusal can be made native model behavior.

Output-side interventions. Self-Refine [Madaan et al., 2023] uses the model itself to critique and revise its outputs. Our S5 strategy applies this idea to scope enforcement: after generating a response, the model is asked whether the response was on-topic and revises if not.

Off-topic detection benchmarks. Most directly related to our work is Chua et al. [2024, ICLR 2025], who introduced the off-topic prompt detection problem and released the gabrielchua/off-topic dataset we use as our primary benchmark. Their work evaluates only fine-tuned bi-encoder and cross-encoder classifiers, not internal-representation methods or fine-tuned LLM behavior. Our comparative study extends their benchmark with a wider range of intervention strategies.

4. Methodology

4.1 Experimental Setup

Model

All strategies use **Llama-3.1-8B-Instruct** - a 32-layer transformer with 4,096-dimensional hidden states. Open-source, fits on a single A100 in fp16, and large enough that probing and fine-tuning experiments are representative of production deployments.

Dataset

We use the **gabrielchua/off-topic** dataset [Chua et al., 2024], which contains 2.62 million labeled (system prompt, user query, off-topic flag) triples. We sample a 5,000-example training set with enforced 50/50 class

balance and domain diversity (8–10 examples per unique system prompt) and a 500-example evaluation set with 50/50 balance. **All 500 system prompts in the evaluation set are completely disjoint from the training set**, ensuring evaluation measures genuine generalization rather than memorization of domain-specific patterns. For strategies requiring LLM-generated responses (S1, S5), responses are generated offline by Llama-3.1-8B with no guardrails. We deliberately avoided GPT-4o for response generation because a well-aligned model would refuse most off-topic queries, producing unrealistic training data.

Evaluation metrics

All strategies are evaluated on the same 500-example test set: guardrail accuracy, F1, false positive rate (FPR; legitimate queries wrongly blocked), false negative rate (FNR; off-topic queries wrongly let through), and inference latency on an NVIDIA A100-SXM4-40GB. Compliance labels (COMPLIED/REFUSED) are produced by GPT-4o-mini following the methodology of Chua et al. (2024).

Infrastructure

Google Colab Pro with A100 GPU access, HuggingFace Transformers (v5.0) and PEFT for model loading, scikit-learn for probe training, and the OpenAI API (GPT-4o-mini) for the LLM-as-judge component. Total external API spend across the project is under \$10. All code, notebooks, datasets, and result artifacts are available at [the project repository](#).

4.2 Strategy 1 (S1): Text Classifier

S1 is a two-stage classifier system that mirrors the symmetric structure of the guardrail problem itself. An input classifier decides whether a user query is in scope before any LLM call is made; an output classifier decides whether a generated response constitutes substantive compliance after the LLM has produced it. Both are binary classifiers fine-tuned independently from roberta-base (125M parameters), chosen for its training stability under bf16 mixed precision and strong default performance on sentence-pair tasks. The motivation for stacking them is that they fail in different ways: the input classifier can miss disguised off-topic queries, while the output classifier can sometimes catch those misses post-hoc by recognizing that the LLM responded substantively to a request it should have refused.

Both classifiers tokenize their inputs as sentence pairs with longest_first truncation to 512 tokens, which preserves the user query intact and trims the system prompt only when necessary. The input classifier consumes (system_prompt, user_query) and predicts off-topic vs. on-topic; the output classifier consumes (system_prompt, user_query, llama_response) and predicts COMPLIED vs. REFUSED, with the response in segment B. Output-classifier labels come from GPT-4o-mini following Section 4.1, which has 90.4% agreement with manual review and establishes a soft ceiling on absolute classifier accuracy. Training data is imbalanced (81% COMPLIED, 19% REFUSED), so we use class-weighted cross-entropy; otherwise both classifiers share hyperparameters: AdamW (lr 2e-5, weight decay 0.01), cosine schedule with 10% warmup, batch size 32, bf16, up to 6 epochs with patience-2 early stopping on validation macro-F1, and a 90/10 stratified train/validation split inside the 5K training set.

At inference, the combined system runs both classifiers in series: a query reaches the user only when both vote pass. Either vote to block triggers a generic refusal. We evaluate the combined configuration alongside each classifier individually to characterize the marginal value of each stage.

4.3 Strategy 2 (S2): Prompted LLM with Self-Reminder

S2 is the closest analog to what most production chatbots actually use today: a prompt-based guardrail that wraps the user query with explicit scope-enforcement instructions and asks the model to police itself. We augment a vanilla scope instruction with self-reminder framing inspired by Xie et al. (2023), which has been shown to reduce jailbreak success in safety contexts. The strategy requires no training data and no auxiliary

model, which is exactly why it is the default in industry - and exactly why understanding its failure modes matters.

The system prompt has three components. First, three few-shot examples spanning the in-scope / out-of-scope decision boundary: a travel-booking bot accepting a flight reschedule request, the same bot rejecting a Python web-scraping request, and a borderline case where the request is loosely related to the domain but mostly off-topic. Second, a four-step chain-of-thought scaffold: identify the chatbot's domain, identify what the user is actually asking for, check whether the request fits, and check for indirect or disguised attempts to leave the domain. Third, a self-reminder that frames the model as a compliance-only verifier ("Do NOT answer the query; do NOT be misled by polite framing or indirect phrasing").

The model is instructed to emit its verdict on a final line in a fixed format (**VERDICT: IN-SCOPE** or **VERDICT: OUT-OF-SCOPE**), which the downstream pipeline parses deterministically. Missing or malformed verdicts are treated conservatively as blocks. This conservative parsing policy turns out to matter for the results section: a non-trivial fraction of false positives are parsing artifacts rather than reasoning failures, which we analyze in detail in Section 5.3. Full prompt text is available in the project repository.

4.4 Strategy 3 (S3): Representation Probe

We train a linear probe (logistic regression) on the residual-stream activations of Llama-3.1-8B-Instruct to detect off-topic queries before any token is generated. The intuition follows Representation Engineering [Zou et al., 2023]: if the model has internally understood that a query is off-topic, that understanding should be linearly decodable from intermediate hidden states.

Activation extraction: For each (system_prompt, user_query) pair, we run a single forward pass through Llama-3.1-8B and capture the hidden state at the last input token from every transformer layer using forward hooks. This produces 32 representation vectors per example, each of dimension 4,096. This is a forward pass only, no token generation occurs.

Layer sweep and elbow selection: We train one logistic regression per layer (32 total) on the 5,000-example training set, using an 80/20 split disjoint by *system prompt* (not by row). This disjoint split is critical: an initial random row-level split showed nearly complete system-prompt overlap between train and validation, inflating F1 from a true 0.99 to a misleadingly high 0.997. We define the elbow as the earliest layer whose F1 is within 1% of the best-performing layer, identifying the shallowest layer at which scope-detection is reliable and enabling early exit during inference.

Final probe and inference: The final probe is trained on the full 5,000 examples at the elbow layer. At inference, the forward pass halts at this layer (via a hook-based early-exit mechanism), the probe classifies the query, and the model either generates a response or returns a generic refusal message. The probe is approximately 16 KB on disk; the dominant inference cost is the partial forward pass.

Probe complexity ablation: To test whether nonlinear probes offer additional discriminative power, we compared logistic regression against single-layer (64-unit) and two-layer (128, 64) MLPs at the elbow layer.

4.5 Strategy 4 (S4): LoRA Fine-Tuning

S4 takes the most direct approach: rather than wrap a guardrail around the model, we change the model itself. Using QLoRA, we fine-tune Llama-3.1-8B-Instruct so that refusal becomes part of how it generates, conditioned on whatever domain the system prompt declares. There is no external classifier and no extra inference pass - at deployment time, the model simply does the right thing.

Training data: We built the training set from the same 5,000-row sample used elsewhere in the project. For the 2,500 off-topic rows, we used GPT-4o-mini to generate a single-sentence refusal tailored to each row's system prompt; a generic "I can't help with that" would have taught the model the wrong behavior. For the 2,441 on-topic rows where Llama already gave a reasonable answer, we kept its existing response. For the 59

on-topic rows where Llama had wrongly refused, we used GPT-4o-mini to generate a correct helpful response as the target. The result is a balanced 5,000-row supervised set, split 4,750 / 250 for training and validation.

Configuration: QLoRA with 4-bit nf4 quantization, rank $r=16$ and $\alpha=32$, applied to all attention and MLP projection matrices (q, k, v, o, gate, up, down). Effective batch size 16 via gradient accumulation, cosine learning rate schedule peaking at $2e-4$, and early stopping with patience 3 on validation loss. Each example is rendered as a system / user / assistant chat template. Training ran on a single A100 using the Unsloth framework. Final evaluation uses the same 500-row held-out set and the same GPT-4o-mini judge as the rest of the project.

4.6 Strategy 5 (S5): Self-Critique Loop

S5 applies the Self-Refine idea [Madaan et al., 2023] to scope enforcement: rather than guarding inputs or representations, give the model a second chance to examine its own output before delivery. We split inference into two passes. In the first pass, the model is given the user's query under the original chatbot system prompt and generates an initial response as it normally would. In the second pass, we re-feed that response under a different system prompt that re-casts the model as a domain checker - its task is no longer to answer the user but to assess whether the response it just produced is consistent with the original chatbot's domain. The model emits a single decision (**IN-SCOPE** or **OUT-OF-SCOPE**), which we parse as the final guardrail verdict; malformed outputs count as parse errors and are conservatively treated as blocks.

The critique prompt is the obvious knob to tune, so we tested three variants on a balanced 100-example subset of the eval set. **v1_simple** describes the role and asks for a single decision. **v2_strict** adds explicit pressure toward refusal ("when in doubt, OUT-OF-SCOPE"). **v3_checklist** decomposes the decision into a structured checklist (domain match? task type match? indirect attempt?) and asks for a verdict only after all boxes are filled. We compare all three on accuracy, precision, recall, F1, false-positive rate, and parse-error counts, using the same GPT-4o-mini judging pipeline as the other strategies.

S5 is interesting precisely because it is post-generation: the unsafe response is fully produced before the guardrail runs. This bounds the strategy's value (it cannot prevent generation, only suppress delivery) but also makes it a natural complement to pre-generation guardrails for defense in depth.

5. Results

5.1 Main Comparison

The table summarizes guardrail performance across all completed strategies on the held-out 500-example evaluation set. Compliance-based metrics (whether the chatbot's response actually complied with off-topic requests) are judged by GPT-4o-mini following Chua et al.'s methodology.

Comparison of guardrail strategies on the shared evaluation set (n=500).

Strategy	F1	Accuracy	FPR	Off-topic Compliance	On-topic Compliance	Notes
Baseline (no guardrail)	0.348	0.588	0.044	0.780	0.956	Reference; complies with 78% of off-topic queries
S1: Text Classifier	0.964	0.964	0.044	0.028	0.956	RoBERTa-base; input alone; combined config underperforms
S2: Prompted LLM	0.934	0.984	0.124	0.124	0.876	Strong recall, weak precision, high latency
S3: Representation Probe (L8)	0.980	0.980	0.024	0.016	0.976	Linear probe; early exit

S4: LoRA Fine-Tuning	0.980	0.980	0.028	0.012	0.972	Strong guardrail performance
S5: Self-Critique (v1)	0.830	0.850	0.06	0.240	0.940	Self-correction improves precision, doubles inference

5.2 Strategy 1 (S1): Text Classifier (Detailed Results)

Headline numbers: We evaluated three configurations on the 500-example test set: input classifier alone, output classifier alone, and the combined two-stage pipeline. The input classifier alone is the strongest configuration, with F1 = 0.964, accuracy = 0.964, and FPR = 0.044 on compliance-based metrics (TP = 243, FN = 7, TN = 239, FP = 11). The classifier itself achieves ROC-AUC = 0.991, indicating near-ceiling ranking quality; the modest FPR is the cost of catching 97.2% of off-topic queries at the default 0.5 threshold. Compared to the unguarded baseline of 78% off-topic compliance, the input classifier reduces this rate to 2.8%, a 28x reduction at the cost of refusing 4.4% of legitimate queries.

Output classifier alone is functionally equivalent to the unguarded baseline: With F1 = 0.351 and accuracy = 0.570, the standalone output classifier is barely distinguishable from the unguarded model (baseline F1 = 0.348, accuracy = 0.588). This is the expected behavior of a labeling-only guardrail: the classifier accurately reproduces Llama’s existing refusal pattern but does not change it. The off-topic compliance rate of 76.8% reflects Llama’s vanilla behavior, not a classifier failure. This finding confirms that an output classifier without an intervention rule provides no guardrail benefit, and motivates the combined configuration below.

The combined system underperforms input alone: Stacking the two classifiers in series degrades guardrail accuracy from 0.964 to 0.928, despite reducing off-topic compliance slightly (2.8% to 2.4%). The mechanism is base-rate asymmetric: the output classifier rescued only 1 of the 7 off-topic queries the input classifier had missed (small gain), while wrongly blocking 19 of the 239 on-topic queries the input classifier had correctly let through (large cost). The output classifier’s residual false-positive rate of roughly 8% on legitimate responses generates more new errors on the much larger on-topic pool than it eliminates from the small misses pool. We conclude that for this dataset and architecture, the input classifier alone is the deployable S1 configuration, and we use it as the headline S1 entry in the Main Comparison Table. The table below summarizes the three configurations side by side.

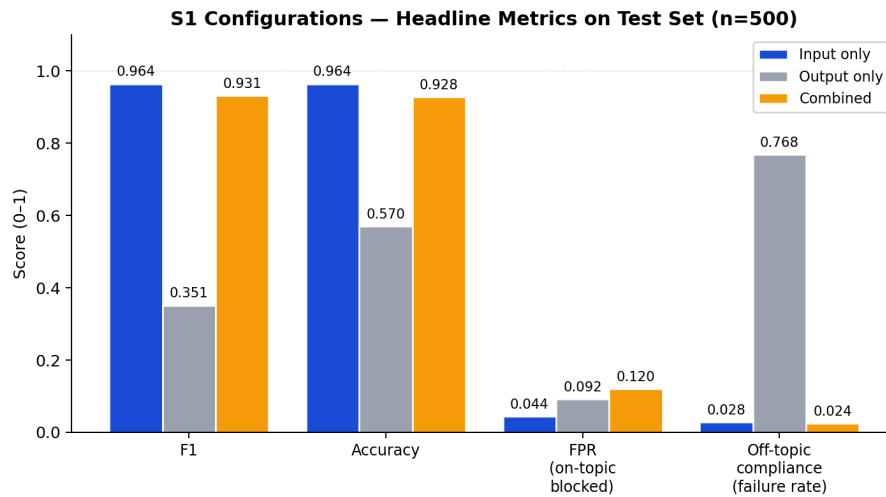
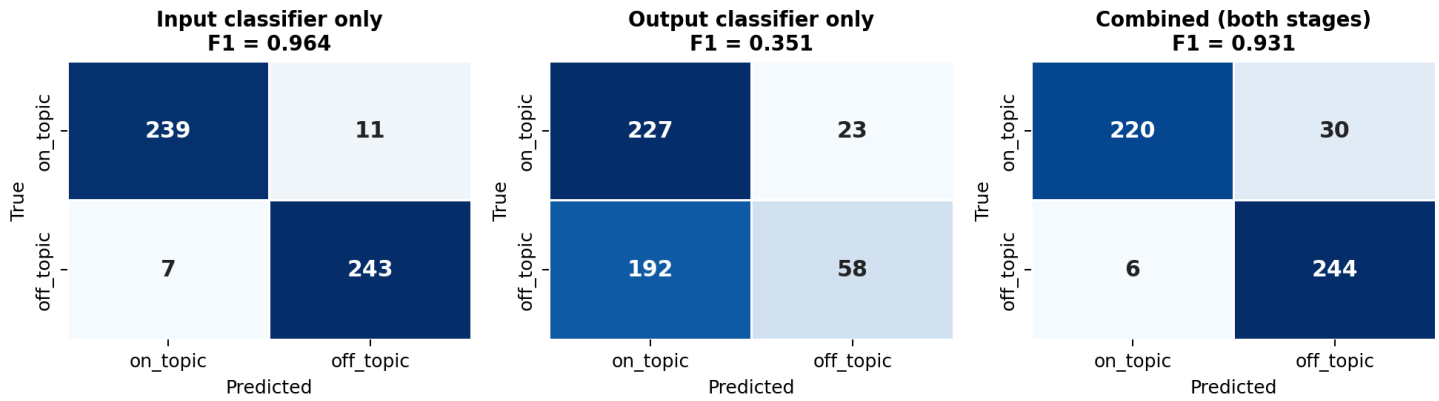
S1 configurations on the shared evaluation set (n=500)

Configuration	F1	Accuracy	FPR	Off-topic Comply	On-topic Comply
Input classifier only	0.964	0.964	0.044	0.028	0.956
Output classifier only	0.351	0.570	0.092	0.768	0.908
Combined (both stages)	0.931	0.928	0.120	0.024	0.880

Comparison with S2 and S3:

S1 (input alone) achieves F1 = 0.964 versus S3’s 0.980 and FPR = 0.044 versus S3’s 0.024, a consistent gap of approximately 1.5 to 2 percentage points across all metrics. Both approaches operate before LLM generation. The gap supports the central hypothesis that internal-representation methods outperform surface-level text classifiers, but the absolute size of the gap is much smaller than the gap between either method and the unguarded baseline. Compared to S2 (prompted LLM), S1 achieves a much lower FPR (0.044 vs 0.124) at comparable F1 and runs orders of magnitude faster, since the inference path is a single RoBERTa-base forward pass rather than a chain-of-thought generation by Llama-3.1-8B. The 7 false negatives include adversarial paraphrases such as a customer-support bot asked “I’m thinking about a career change, what do you think about machine learning roles?”, which suggests that the gap to S3 may be partly attributable to limited training-set coverage of such patterns rather than a fundamental limitation of text-based classification.

S1 Configurations — Confusion Matrices on Test Set (n=500)



S1 Performance for input / output / combined classifier on the 500-example test set

5.3 Strategy 2: Prompted LLM (Detailed Results)

Headline numbers: S2 achieves accuracy = 0.984 and F1 = 0.934. The confusion matrix reveals a clear asymmetry: of 250 off-topic queries, only 4 were wrongly passed (FNR = 0.016); of 250 in-scope queries, 31 were wrongly blocked (FPR = 0.124). S2 is a high-recall, low-precision guardrail - it almost never misses an off-topic query, but over-refuses legitimate users at five times the rate of S3.

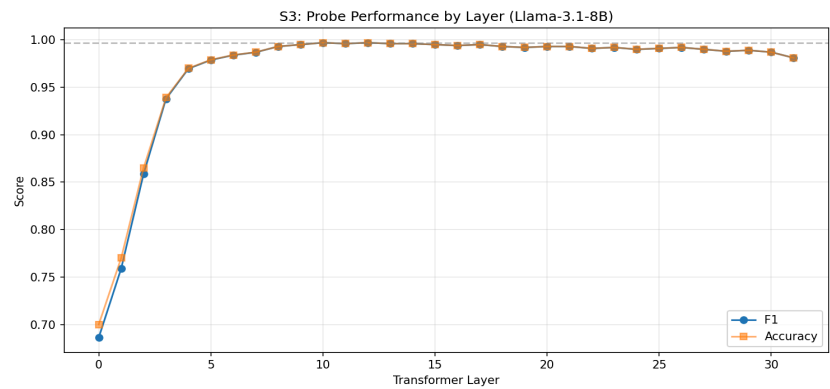
Parse-error analysis: A key implementation finding: 26 of the 31 false positives (84%) are parsing artifacts rather than reasoning errors. In these cases, the model's chain-of-thought correctly concluded IN-SCOPE but reproduced both verdict options from the prompt template, causing the parser (which checks for OUT-OF-SCOPE before IN-SCOPE) to return a false block. Only 5 of the 31 false positives reflect genuine reasoning failures; S2's true reasoning-level FPR is approximately 2.0% (5/250), closely comparable to S3's 2.4%. The 5 genuine misclassifications and the 4 false negatives share a common boundary-ambiguity pattern: a financial advisor bot blocking a query mentioning "moving money offshore" because of the word "secretly"; an environmental educator passing through a query about applying for an offshore drilling permit because of plausible domain connection.

Latency and comparison with S3: S2's average latency of 11,190 ms is the highest of all strategies, reflecting the cost of generating a full chain-of-thought trace before emitting the verdict. Despite nearly identical accuracy to S3 (0.984 vs. 0.980), S2's F1 of 0.934 trails S3's 0.980 due to higher reported FPR, and it runs approximately 890× slower (11,190 ms vs. 12.6 ms). Prompting is zero-cost to deploy and requires no training data, but its reliance on output-formatting stability at inference time introduces both parsing fragility and significantly higher latency.

5.4 Strategy 3: Representation Probe (Detailed Results)

Layer sweep:

The figure shows F1 across all 32 transformer layers. Performance improves rapidly through layers 0–8, plateaus from layer 8 onward, and shows no meaningful gain in deeper layers. The elbow layer is **layer 8** (F1 = 0.993 on the disjoint validation set); the best-performing layer is layer 10 (F1 = 0.997).



F1 by transformer layer (system-prompt-disjoint validation split).

Final evaluation:

The probe trained at layer 8 achieves F1 = 0.982, accuracy = 0.982, AUC = 0.993, FPR = 0.020 (5/250 in-scope queries blocked), and FNR = 0.016 (4/250 off-topic queries let through) at the probe-prediction level. End-to-end with the gating wrapper and GPT-4o-mini compliance labeling, the corresponding compliance-based metrics are F1 = 0.980, accuracy = 0.980, FPR = 0.024, off-topic compliance rate = 0.016, on-topic compliance rate = 0.976. The two metric spaces yield nearly identical results, confirming that S3's deployed guardrail behavior is essentially a deterministic function of the probe's decision rather than a post-hoc generation artifact. Compared to the unguarded baseline (78% off-topic compliance), S3 reduces off-topic compliance by **48×** while leaving on-topic compliance essentially intact (96% to 98%).

Latency benchmark:

Over 50 runs on an A100-SXM4-40GB after warmup, a full forward pass takes 38.89 ± 1.03 ms; early-exit at layer 8 plus probe inference takes 12.59 ± 1.48 ms: a **3.09×** speedup, **67.6% latency reduction**.

Probe architecture ablation at layer 8.

Probe	F1	Accuracy	AUC	Parameters
Logistic Regression	0.9820	0.9820	0.9930	4,097
MLP (64 hidden)	0.9781	0.9780	0.9930	262,338
MLP (128, 64 hidden)	0.9839	0.9840	0.9920	532,802

Logistic regression matches the larger two-layer MLP within 0.002 F1 while using 130× fewer parameters; the single-layer MLP is marginally worse than the linear probe, likely due to overfitting. Identical AUC across all three architectures (0.992–0.993) indicates probes rank examples in the same order. We interpret this as strong evidence that the off-topic feature is linearly decodable from Llama-3.1-8B's layer-8 residual stream, consistent with prior findings in representation engineering [Zou et al., 2023].

Error analysis: All 9 misclassifications on the evaluation set are genuine boundary cases. Examples include a data-privacy compliance bot asked about avoiding legal penalties for breaches (labeled off-topic; probe predicted in-scope, arguably correct), a movie production assistant asked for a script outline, and an emotional-wellness advisor asked for medical advice on headaches. The probe's errors track defensible interpretations of system-prompt scope rather than systematic failures.

5.5 Strategy 4: LoRA (Detailed Results)

S4 performs strongly. Off-topic compliance drops from the unguarded baseline's 78% to 1.2%, a 65× reduction, while on-topic compliance actually improved slightly, from 95.6% to 97.2%. Guardrail accuracy lands at 0.98 with F1 = 0.98, putting S4 in a near-tie with S3 on the leaderboard.

Two things stand out qualitatively.

First, the fine-tuned model's refusals are domain-aware: when a travel bot is asked to write Python, it doesn't just say "I can't help" - it says something like "I'm a travel assistant, so I can't help with code, but I'd be happy to plan your trip." The behavior was learned from the GPT-4o-generated training targets and emerges naturally at inference.

Second, the FPR actually improved over the unguarded baseline (0.044 to 0.028), meaning the model didn't over-correct into refusing legitimate queries, meaning the model didn't over-correct into refusing legitimate queries. The trade-off, of course, is that S4 is the only strategy that requires retraining whenever the deployment domain changes. The probe in S3 can be retrained in seconds; the LoRA adapter takes hours.

5.6 Strategy 5: Self-Critique Loop (Detailed Results)

Critique-prompt comparison (n=100 balanced).

Prompt	Accuracy	Precision	Recall	F1	FPR	Missed OOD	Parse Errors
v1_simple	0.85	0.93	0.76	0.83	0.06	0.24	7
v2_strict	0.61	0.63	0.54	0.58	0.32	0.46	44
v3_checklist	0.68	0.95	0.38	0.54	0.02	0.62	33

v1_simple has the best overall performance, balancing accuracy (0.85), F1 (0.83), and a low false-positive rate (0.06). v2_strict performs worst (accuracy 0.61, F1 0.58, FPR 0.32) because the strictness inflates over-refusal. v3_checklist achieves the highest precision (0.95) but extremely poor recall (0.38), missing many off-topic queries. The high parse-error counts for v2 and v3 (44 and 33 of 100) indicate that overly structured prompts also degrade output-format reliability. S5 also incurs roughly 2× the inference cost of single-stage prompting because every query requires both an initial response and a critique pass; this overhead is partly offset by the model's ability to correct itself in clearly off-domain cases (e.g., programming or creative-writing requests it initially accepts but later refuses).

5.7 Discussion of Findings

1. Probing the model's representations beats asking it to police itself:

S3's compliance F1 of 0.980 with a 2.4% false-positive rate cleanly beats S2's 0.934 with 12.4%, and both run on the same base model. The starker comparison is against the unguarded baseline, where Llama complies with 78% of off-topic queries: S3 drops that to 1.6% - a 48× reduction - while only blocking 2.4% of legitimate users. The methods do roughly the same job, but where they intervene matters a lot. Reading what the model has already understood (a vector at layer 8) is more reliable than asking it to follow instructions about what to refuse.

2. Domain-scope is a linearly decodable feature in middle layers:

The MLP ablation shows logistic regression matches deeper architectures at layer 8, and the layer sweep shows reliable detection emerges by layer 8 of 32 (25% of total depth). This connects to the broader Representation Engineering literature: high-level concepts including domain consistency are encoded as linear directions in transformer activations.

3. Early exit yields significant inference savings:

S3 runs 3.09× faster than the baseline forward pass while matching or exceeding alternative guardrails on accuracy. For latency-sensitive deployments, this is a meaningful engineering result.

4. Pre-generation guardrails are easier to reason about:

A small but methodologically interesting finding fell out of running S3 through both the probe-level and compliance-based metrics: they agreed almost exactly ($F1 = 0.982$ vs. 0.980). That's because the probe's decision deterministically drives the refusal - there's no generation step in between to introduce noise. For S2 and S5, the chain from "guardrail decision" to "user-facing response" is longer, and stochasticity in generation widens the gap. If you care about auditability, this is a real argument for putting the guardrail before generation rather than after.

5. Self-critique provides moderate improvements but is highly prompt-sensitive:

S5's best variant (v1_simple, $F1 = 0.83$) underperforms both S2 and S3, and the gap between variants ($F1$ 0.54–0.83 across three prompt designs) shows that small wording changes meaningfully affect guardrail behavior. As a post-generation strategy, S5 also cannot prevent the initial unsafe response from being generated; it can only suppress it before delivery, at the cost of doubling inference latency.

6. Surface text classification gets surprisingly close to internal methods: Our hypothesis predicted internal methods (S3, S4) would clearly beat surface methods (S1, S2). The data partially confirms this. S3 and S4 do achieve the best $F1$ (0.980), but a well-designed text classifier (S1) reaches 0.964, only 1.6 points behind. The bigger axis is pre-generation versus post-generation: every pre-generation guardrail (S1, S3, S4) reaches $F1 \geq 0.96$, while methods that depend on inference-time reasoning by the LLM (S2 prompted, S5 self-critique) lag by 5 to 15 points. The hypothesis was right about the direction of the effect but understated how much a careful text classifier can recover.

6. Conclusion and Future Work

We presented a systematic comparison of five guardrail strategies for domain-constrained LLM chatbots, operating at different levels of the LLM pipeline. Our central finding is that a linear probe on intermediate transformer activations matches or exceeds the accuracy of more invasive interventions while running 3× faster than the baseline model. The off-topic distinction appears to be a linearly decodable feature of Llama-3.1-8B's residual stream by layer 8 of 32, consistent with the Representation Engineering hypothesis that high-level semantic features are linearly encoded in LLM activations.

The engineering implication is concrete. A 4,000-parameter logistic regression - small enough to fit in 16 KB of disk space - sitting alongside the base model is enough to drive off-topic compliance from 78% down to 1.6%, while leaving the model's normal behavior on legitimate queries essentially untouched. Prompt engineering (S2) gets to similar overall accuracy, but it does so by over-refusing real users at five times the rate, and it costs roughly 890× more wall-clock time per query. The early-exit trick - stopping the forward pass at layer 8 instead of running the full 32 - gives a further 67% latency reduction with no measurable accuracy cost. For latency-sensitive deployments, this is the kind of result that actually changes a deployment decision.

Among the surface-level methods, a fine-tuned text classifier (S1) closed most of the gap to internal methods at $F1 = 0.964$, suggesting that careful supervised training on the right labeled data is a strong baseline that any internal-method paper should beat by a meaningful margin.

Limitations: Our evaluation set, while disjoint from training, is drawn from the same synthetic distribution (gabrielchua/off-topic). Real-world adversarial robustness - particularly to multilingual jailbreaks and prompt injections - was not stress-tested in depth. Our probe was trained and evaluated on Llama-3.1-8B only;

cross-model generalization remains an open question. Our setup uses single-turn queries; multi-turn conversational scope erosion is a known failure mode that none of our strategies explicitly defend against.

Future work:

Promising directions include:

- (a) **multilingual stress testing** - translating the eval set into low-resource languages (Zulu, Bengali, Hindi) to test whether internal-representation methods inherit the multilingual understanding the base model already possesses;
- (b) **cross-model probe transfer** - training on Llama-3.1-8B and evaluating on Phi-3-mini activations;
- (c) **multi-turn scope erosion** - constructing conversation traces with gradual off-topic drift;
- (d) **composing strategies** - combining S3 (cheap input filter) with S5 (output critique) for defense in depth.

References

- [1] Bai, Y., Kadavath, S., Kundu, S., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073.
- [2] Chua, G., Chan, S. Y., & Khoo, S. (2024). A Flexible Large Language Models Guardrail Development Methodology Applied to Off-Topic Prompt Detection. ICLR 2025. arXiv:2411.12946.
- [3] Deng, Y., Zhang, W., Pan, S. J., & Bing, L. (2024). Multilingual Jailbreak Challenges in Large Language Models. ICLR 2024. arXiv:2310.06474.
- [4] Hu, E. J., Shen, Y., Wallis, P., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022. arXiv:2106.09685.
- [5] Inan, H., Upasani, K., Chi, J., et al. (2023). Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674.
- [6] Lasso Security (2026). Amazon Chatbot Gone Wrong: Rufus Jailbreak Disclosure. lasso.security/blog/amazon-chatbot-gone-wrong
- [7] Madaan, A., Tandon, N., Gupta, P., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023. arXiv:2303.17651.
- [8] OWASP Foundation (2025). 2025 Generative AI Incident & Exploit Roundup. genai.owasp.org/2025/03/06/owasp-gen-ai-incident
- [9] Xie, Y., Yi, J., Shao, J., et al. (2023). Defending ChatGPT against Jailbreak Attack via Self-Reminders. Nature Machine Intelligence. arXiv:2310.06117.
- [10] Yong, Z. X., Menghini, C., & Bach, S. H. (2023). Low-Resource Languages Jailbreak GPT-4. arXiv:2310.02446.
- [11] Zou, A., Phan, L., Chen, S., et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv:2310.01405.